

颜正浩

741699287@qq.com | Android 开发工程师 13261010889

教育经历

南京理工大学紫金学院

本科, 计算机科学与技术
2014.09-2018.06

开源及分享

博客: Catheryan-Blog
Github:// catheryan
yuque:// 知识分享

技能

Android

主导多个大型项目模块
上线

语言

Kotlin ·
Java ·
C++ ·
Javascript

跨端技术

Kotlin Multplatform
·
Rax · Vue

DevOps

gradle · ci·cd
Jenkins · 自动化测试

从业经历

腾讯·OTT 视频业务 实习

2017.06 - 2017.09 | 深圳

- 合作一份实用性专利 已归档

阿里巴巴·大文娱 优酷·基础业务 Android 开发工程师

2018.06 - 2021.01 | 北京

- 主要负责首页基础业务以及性能优化
- 期间多次获得高潜评价

哔哩哔哩·商业 Android 开发工程师

2021.02 - 至今 | 上海

- 商业化交易和票务对接人, 后续负责广告业务
- 主要负责需求工程设计以及技术攻关

项目经历-核心

动态化落地 Owner 基于 redwood 的跨端动态化框架

2025.03 - 2025.05

- 项目背景:
 - 线上业务迭代周期长, 运营成本高, 需提升效率与速率。
 - 部门搭建可视化组件平台, 增强运营与扩展能力。
 - 已具备丰富 KMP 跨端能力, 助力平台化和多端发展。
- 方案对比:
 - 复用 KMP 组件库, 支持快速页面搭建, 实现可视化平台。
 - 利用 KMP 多端能力, 统一 Kotlin 代码, 输出 JS/C++ 产物, 低成本支持多端。
 - 对比 RN 与 Redwood KMP 方案, 最终采纳 Redwood, 完成性能与帧率测试。
- 项目成果:
 - 从 0 到 1 完成动态体系搭建, 实现远端分发、CI/CD 及热更新。
 - 初步搭建可视化组件库, 支持业务自助页面配置。
 - 业务上线周期由一周缩短至 1-2 天, 迭代与实验效率显著提升。
- 个人贡献:
 - 主导项目整体设计与落地, 推动项目进展。
 - 输出 KMP 沙盒工程、Zipline 集成及业务容错方案。
 - 搭建并规范跨端 UI 组件库, 确保复用与一致性。

跨端链式事务处理 Owner 基于 KMP 的链式点击处理框架

2024.12 - 2025.02

- 项目背景:
 - 广告业务在多端 (如 iOS、Android 等) 存在点击逻辑分散、过程开发重复离散的问题, 导致双端坑位开发成本高、效率低。
 - 亟需通过统一方案减少端间重复开发, 实现广告点击节点逻辑的高效复用与管理。
- 方案对比:
 - 传统方案: 各端独立实现广告点击节点, 导致坑位重复开发、维护成本高, 且易出现逻辑不一致。
 - 新方案: 引入统一的广告点击节点链式处理体系, 采用分层设计, 实现多端逻辑统一, 支持节点灵活组合及跨端接口差异化处理。
- 项目成果:
 - 成功实现多端广告点击逻辑的统一节点管理, 显著提升复用率。
 - 双端坑位开发时间节约近 80%, 提高整体开发效率和业务迭代速度。
- 个人贡献:
 - 作为项目 Owner, 主导设计并落地核心链式处理体系及全生命周期分层架构。
 - 负责核心链式处理框架的开发, 设计节点灵活组合机制, 以及实现跨端差异化接口适配。

InAppMessage Owner 基于轮训结构的站内信系统

2021.11 - 2022.01

- **项目背景:**
 - 交易业务需要高效的站内信触达通道，没有弹窗和浮层触达方案，影响用户体验和转化。
 - 后台长链接通道未搭建，现有方案无法满足高并发和高可用的需求。
- **方案对比:**
 - 常用方案：需要构建长链接通道，开发和维护成本高，且易受网络环境影响，稳定性和兼容性存在挑战；长链接通道后台暂未搭建
 - 新方案：采用优化轮训结构，结合 `aidl` 和 `atomic` 原语，提升消息处理效率与系统稳定性，简化跨进程通信和资源管理。
- **项目成果:**
 - 成功搭建基于轮训的站内信系统，显著提高交易业务触达效率和购买转化率。
- **个人贡献:**
 - 作为项目 Owner，主导整体方案设计与落地，负责前期方案选型。
 - 设计轮训消息体系，利用 `aidl` 和 `atomic` 解决跨进程通信及资源竞争难题。
 - 后期对于策略，也做了端智能调研相关工作，主要运用 `tensorflow lite` 进行模型推理，提升端侧智能化能力。

TradeSdk Owner 基于 gradle 的交易流程工程化

2023.07 - 2023.9

- **项目背景:**
 - 已有多渠道交易流程是 web 流程，需要维护多套，影响整体交易体验和开发效率。
 - web 存在体验较差以及功能落后等问题，需要推动 Native 工程化，提升交易流程的稳定性和可维护性。
- **方案考虑点:**
 - 大仓流程定制化严重，缺乏统一工程化 sdk 标准，且各宿主渠道底层库差异较大
 - Native 工程化需要考虑多渠道的差异化配置和路由处理，确保各渠道流程的稳定性和一致性。
- **项目成果:**
 - 平稳实现各渠道交易正逆向流程的 Native 工程化，节约原有开发人力成本 (1hc)，显著提升交易体验与迭代效率。
 - 提供一套大仓的交易流程工程化 sdk 解决方案，支持多渠道的差异化配置和路由处理，沉淀了一套可复用的工程化 sdk 插件能力
- **个人贡献:**
 - 作为项目 Owner，主导流程分层设计和整体线上路由方案配置进行过渡，做到有问题及时回滚。
 - 采用分层设计优化路由 Config 迁移，gradle 插件实现分层解耦以及渠道环境变量配置，便于业务根据渠道区分维护与扩展。

半屏容器栈 Owner 基于 Fragment 的交易容器栈

2023.02 - 2023.04

- **项目背景:**
 - 交易流程需要灵活投放到不同业务场景，原有架构对扩展和业务对接支持有限，缺乏统一技术基建。
 - 交易现有的正向和逆向流程分散在不同业务方，需要进行复用和整合，提升业务对接效率。
- **方案对比:**
 - 原有方案：页面能力分散，路由及事务处理缺乏统一框架，业务方接入成本高。
 - 新方案：搭建容器栈，统一支持 native 面板及 web 面板，复用 fragment 页面能力和路由体系，提供标准化容器事务处理能力；
 - 需要设计多种 panel 的适配方案，支持 web 和 native 面板的灵活投放。
- **项目成果:**
 - 成功落地容器栈，支持交易流程灵活投放任意场景，提升业务对接效率。
 - 半屏容器方案为后续业务方提供高复用技术基建，提升整体对外业务对接效率。
- **个人贡献:**
 - 作为项目 Owner，主导容器栈方案设计与落地，实现 native 和 web 面板搭载，负责整体对外业务对接。
 - 复用原有 fragment 页面业务能力和路由体系，设计并实现容器事务处理能力，提升系统扩展性与可维护性。

Flutter 实验 Owner 基于 Flutter 的落地页

2020.04 - 2020.06

- **项目背景:**
 - 多端页面开发成本高，业务需求快速迭代，亟需统一跨端页面能力与组件生态，提升开发效率与复用率。
- **方案对比:**
 - 原有方案：各端独立开发，组件生态割裂，工程能力不统一，持续集成能力有限。
 - 新方案：引入 Flutter 跨端能力，接入阿里 atalss 工程能力，统一基础组件及工程能力，实现多端高效协同。
- **项目成果:**
 - 主要利用 Flutter 的跨端能力，搭建统一页面能力，提升业务开发效率与组件复用率。
- **个人贡献:**
 - 作为贡献者，参与 Flutter 生态搭建，包括基础组件开发与持续集成（CD）能力建设。
 - 负责接入阿里 atalss 工程能力，完善 Flutter 组件生态能力，助力团队多端协同。

首页启动优化 贡献者 基于 DAG 的启动架构

2020.07 - 2020.10

- **项目背景:**
 - 首页启动流程复杂，任务调度与依赖管理不够高效，导致启动耗时较长，影响用户体验。
- **方案对比:**
 - 原有方案：任务依赖关系不清晰，缺乏统一调度机制，启动流程无法精细优化，数据采集和分析能力有限。
 - 新方案：引入 KSP 能力，设计 DAG（有向无环图）任务调度体系，实现任务依赖前置编译检查及耗时监控，优化整体启动流程。
- **项目成果:**
 - 首页启动流程优化，启动耗时由 1.5s 降至 0.8s，显著提升用户体验。
- **个人贡献:**
 - 作为贡献者，利用 KSP 能力进行数据打桩、DAG 设计与实现。
 - 负责 DAG 任务的前置编译检查和 KSP 任务耗时监控，助力流程优化与性能提升。

开源贡献

HoneyBee-Plugins 对路由注解核心机制的拆解

自我评价

- 热爱技术，热爱分享，热爱开源，具备良好的沟通能力和团队协作能力